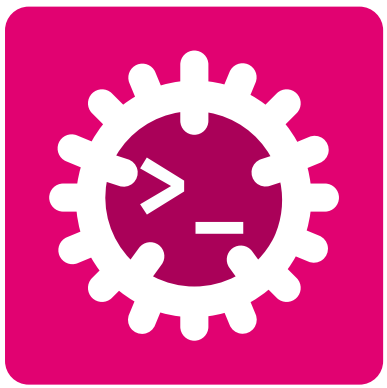
Hes·so VALAIS WALLIS

School of Engineering

SYSTEMS ENGINEERING

INFOTRONICS



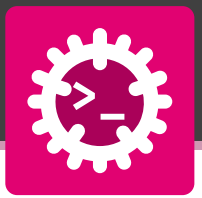
Advanced Programming

Debug // Testing // Documentation

Silvan Zahno

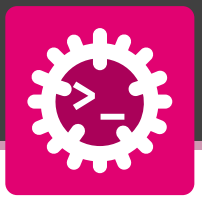
University of Applied Sciences Western Switzerland - HES-SO Valais Wallis
Systems Engineering
Infotronics

2026-08-11 - v0.1.0



Contents

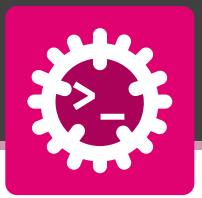
Contents



Debug / Testing / Documentation

Ship code that works - and prove it

Ensuring correctness continuously



The safety net of a healthy Rust project - each layer catches a different class of mistake.

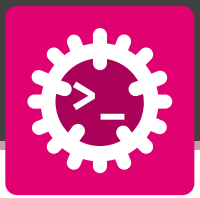
- **Unit tests** - `#[test]` functions, run by `cargo test`
  - Happy path: expected, valid inputs
  - Boundary path: edge cases (empty, min/max, limits)
  - Negative path: invalid inputs and error handling
- **Integration & doc tests** - the `tests/` folder and runnable examples in docs
- **`cargo check` / `clippy`** - fast type-check and lints
- **`cargo fmt`** - one canonical style
- **Error handling** - `Result` / `Option`, no surprise crashes
- **Logging & debugging** - see what the program actually did
- **Profiling** - measure before you optimise
- **CI/CD** - run **all** of the above automatically on every push

The goal: catch a mistake **as early and as cheaply** as possible - ideally at compile time.

Testing

Prove the code does what you claim

Unit tests



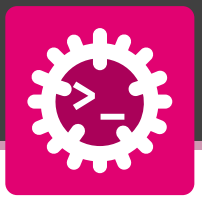
A **unit test** checks **one** piece of code in **isolation** - including the **edge cases**.

```rust
// the code under test
pub fn parse_temperature(s: &str) -> Option<i32> {
    s.trim().trim_end_matches("°C").parse().ok()
}

#[cfg(test)]                    // compiled only for `cargo test`
mod tests {                     // a *submodule* for the tests
    use super::*;               // bring the parent module into scope

    #[test]                     // a *unit test* function
    fn plain_number() {
        assert_eq!(parse_temperature("20"), Some(20));
    }
    #[test]
    fn strips_unit_and_spaces() {
        assert_eq!(parse_temperature(" 20°C "), Some(20));
    }
    #[test]
    fn rejects_garbage() {
        assert_eq!(parse_temperature("hot"), None);
    }
}
```

- lives **next to the code**, in `#[cfg(test)] mod tests`
- `#[test]` marks each test function
- the rhythm: **arrange** ⇨ **act** ⇨ **assert**
- test the **happy path**, the **edge**, and the **failure**
- run them all with `cargo test`

A failure prints the **file, line** and **both values** - it points right at the bug.

Integration & Doc tests



The **same** function, tested two more ways - both run under `cargo test`.

Integration Tests

files in `tests/`, exercise the crate **from outside**

- only public API is visible

```rust
// tests/temperature.rs
use weather::parse_temperature;

#[test]
fn reads_celsius() {
    assert_eq!(parse_temperature("20°C"), Some(20));
    assert_eq!(parse_temperature("nope"),  None);
}
```

Doc tests

Code in `///` examples is **compiled and run** ⇨ documentation that **can't go stale**

```rust
/// Parses a temperature like `"20°C"`.   DOC test
///
/// # Examples
/// ```
/// use weather::parse_temperature;
/// assert_eq!(parse_temperature("20°C"), Some(20));
/// ```
pub fn parse_temperature(s: &str) -> Option<i32> {
    // ...
}
```

Assertions & expected failures

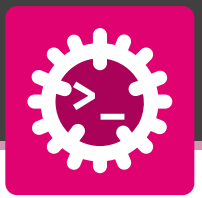
An assertion says what **must** be true - and sometimes the **correct** behaviour is to **panic**.

```rust
// °F → °C, rejecting impossible input
fn f_to_c(f: f64) -> f64 {
    assert!(f ≥ -459.67, "below absolute zero");
    (f - 32.0) * 5.0 / 9.0
}

#[test]
fn freezing_and_boiling() {
    assert_eq!(f_to_c(32.0), 0.0);   // == (both shown on fail)
    assert_ne!(f_to_c(212.0), 0.0);  // ≠ (it is 100.0)
}

#[test]
#[should_panic(expected = "below absolute zero")]
fn rejects_impossible() { f_to_c(-500.0); } // pass if it panics

#[test]
fn parses_ok() -> Result<(), String> {
    let t = parse_temperature("20°C").ok_or("bad")?;
    assert_eq!(t, 20);               // a test may return Result
    Ok(())
}
```

- `assert_eq!` / `assert_ne!` print **both values** on failure
- `assert!(cond, "msg")` - assert a bool, with a **message**
- `#[should_panic]` - the test **passes only if it panics**
- a test may **return `Result`** - `?` inside, an `Err` fails it
- filter: `cargo test freezing` runs all function with the word `freezing` in the funciton name.
  ⇨ Here only `freezing_and_boiling()`

Code Quality & CI

Automate the boring checks



Three commands that catch **most** mistakes before a single test runs.

```
cargo fmt      # one canonical style
cargo check    # type-check only, no binary
cargo clippy   # ~750 lints, catches foot-guns

# auto-fix where possible
cargo fmt --all
cargo clippy --fix
```

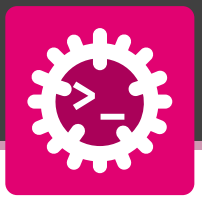
Rust fmt can be configured with `rustfmt.toml` - but the **default** is good enough for most projects. Just run it **on save** in your editor and forget about it.

```
edition = "2024"

tab_spaces = 2
max_width = 160

reorder_imports = true
```

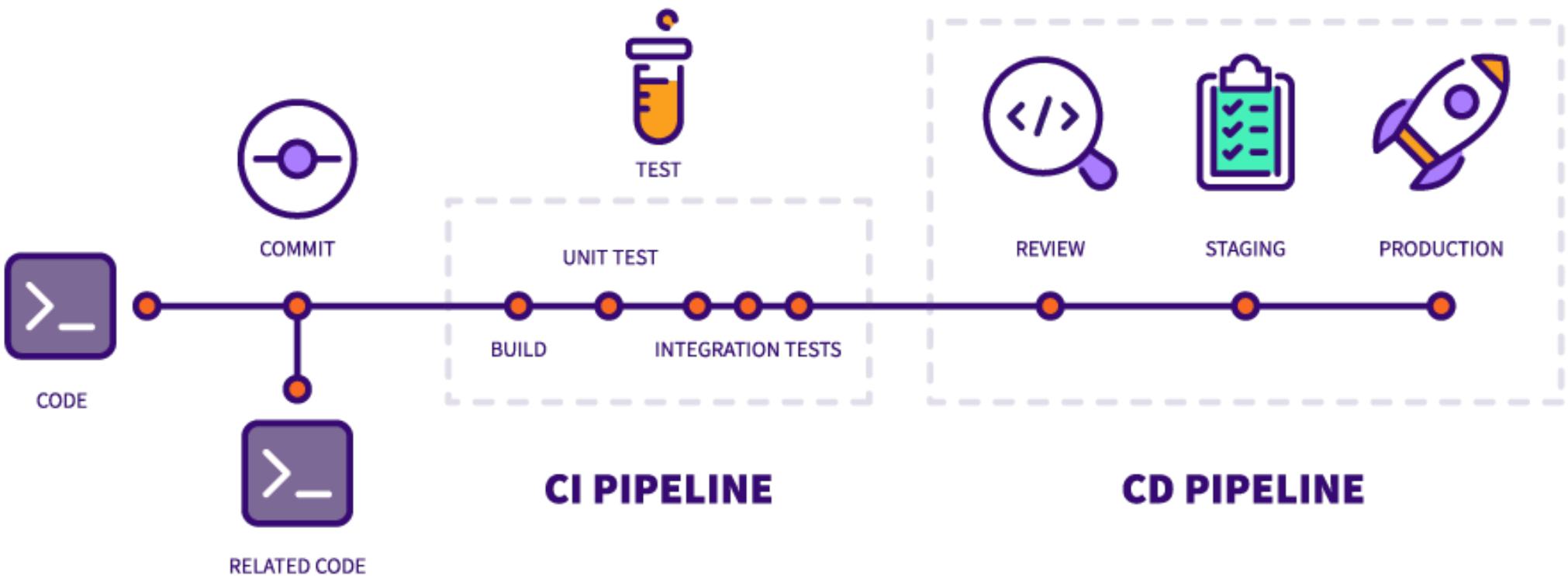
- **fmt** - rustfmt; run it **on save** and end style arguments
- **check** - compiler **front-end** only → fast feedback loop
- **clippy** - idiomatic-Rust **linter**; explains **why** and suggests a fix

> In **CI**, make warnings fatal: `cargo clippy -- -D warnings`.

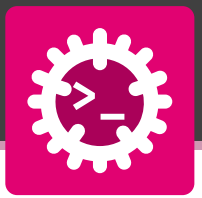
Continuous Integration



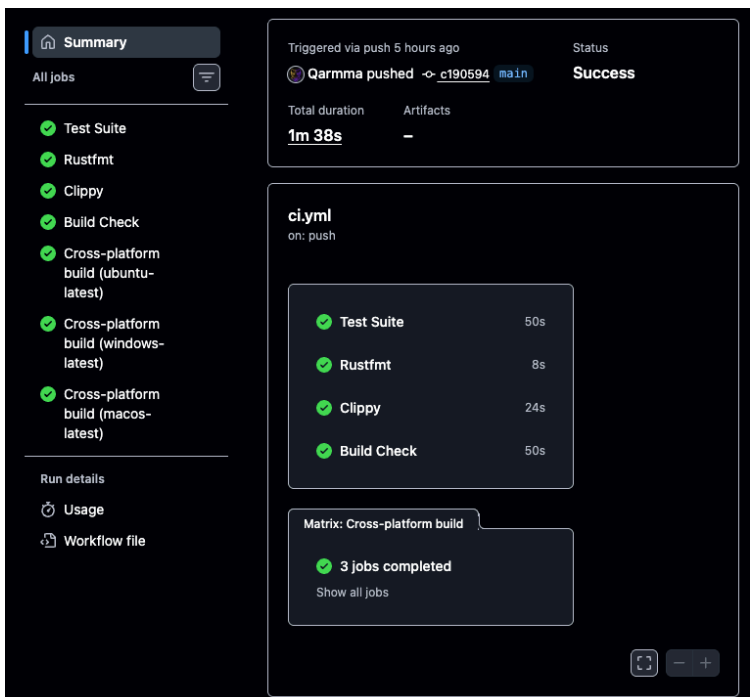
Run every check **automatically** on each **push / pull-request**



Continuous Integration



```yaml
# .github/workflows/ci.yml
on: [push, pull_request]
jobs:
  fmt:
    - run: cargo fmt --all -- --check
  clippy:
    - run: cargo clippy -- -D warnings
  test:
    - run: cargo test --verbose
  build:
    - run: cargo build --verbose
```



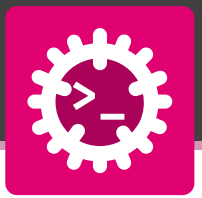
- the **same** commands you run locally - now enforced for **everyone**
- four parallel jobs: **fmt · clippy · test · build**
- on **every push & PR** to `main`

> 
> Green check = formatted, lint-clean, builds, **all tests pass**.

Handling Failures

When things go wrong, gracefully

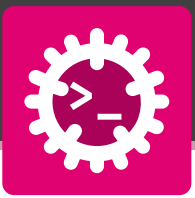
Handling failures: the toolbox



`Result<T, E>` and `Option<T>` make failure **part of the type** (see **Imperative Programming**).

Method	What it does	Use when
Propagate / handle		
`?`	hands the `Err`/`None` to the **caller**	the caller should decide
`match` / `if let`	branch on each case **explicitly**	you handle both paths here
Fallback - never panics		
`unwrap_or(x)`	the value, or `x`	you have a cheap default
`unwrap_or_else(f)`	the value, or `f()`	the default is **expensive** to build
`unwrap_or_default()`	the value, or `T::default()`	`0` / `""` / empty is fine
Panic - give up		
`unwrap()`	the value, or **panic**	tests / “can’t happen”
`expect("msg")`	the value, or **panic w/ msg**	document the invariant
`panic!()`	**abort** the thread	unrecoverable bug

Handling failures: in practice



```rust
use std::fs;
fn load_port(path: &str) -> Option<u16> {
  // ? propagates None upward if read fails
  let text: String = fs::read_to_string(path).ok()?;
  // ? propagates None upward if parsing fails
  let port: u16 = text.trim().parse().ok()?;
  Some(port)
}

fn main() {
  // 1. Fallback - never panics: use 8080 if loading fails
  let port: u16 = load_port("app.toml").unwrap_or(8080);

  // 2. Handle both arms explicitly: success and failure
  if let Some(p) = load_port("app.toml") {
    println!("listening on {p}");
  } else {
    eprintln!("config problem, using defaults");
  }

  // 3. Fallback with a computed/expensive default
  let port: u16 = load_port("app.toml").unwrap_or_else(|| {
    eprintln!("falling back to default port");
    8080
  });

  // 4. Fallback to the type's default value
  let port: u16 = load_port("app.toml").unwrap_or_default();

  // 5. Last resort in a prototype: panic with a message if
config is missing/invalid
  let port: u16 = load_port("app.toml").expect("config
required");

  // 6. Unrecoverable state: abort explicitly
  if port == 0 {
    panic!("port 0 is never valid");
  }
}
```

In the example

- `?` - bubble the error **up**
- `unwrap_or` - safe default `8080`
- `if let` / `match` - handle **both** arms
- `expect` - **panic** w/ message
- `panic!` - **abort** the thread

Exercise - Handling Failures

The code is given. On each `// TODO` line, add the **right method** from the failure toolbox - each line uses a **different** one (`?`, `unwrap_or`, `unwrap_or_else`, `unwrap_or_default`, `expect`).

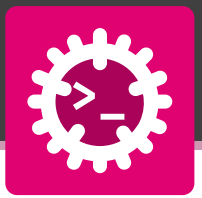
```rust
use std::num::ParseIntError;

// parse_port returns Result<u16, ParseIntError>
fn parse_port(s: &str) -> Result<u16, ParseIntError> {
  s.trim().parse()
}

// TODO 1: propagate the Err to the caller (one operator)
fn sum_ports(a: &str, b: &str)
    -> Result<u16, ParseIntError> {
  let x = parse_port(a)/* ? */; // -> u16
  let y = parse_port(b)/* ? */; // -> u16
  Ok(x + y)
}
```

```rust
fn main() {
  // TODO 2: use 8080 when parsing fails (cheap default)
  let p1: u16 = parse_port("9000")/* ? */;

  // TODO 3: build the default lazily (only if needed)
  let p2: u16 = parse_port("oops")/* ? */;

  // TODO 4: fall back to u16::default() (= 0)
  let p3: u16 = parse_port("oops")/* ? */;

  // TODO 5: panic with a message if it must succeed
  let p4: u16 = parse_port("oops")/* ? */;
}
```

Debugging

Find the bug

Print & inspect



The compiler is your **first** debugger - then reach for a print.

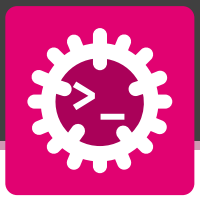
```rust
fn main() {
  let xs = vec![1, 2, 3];
  println!("xs = {xs:?}");   // you choose what to show
  let _n = dbg!(xs.len()) * 2; // prints AND returns the value
}
```

```
Compiling playground v0.0.1 (/playground)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.49s
    Running `target/debug/playground`

[src/main.rs:6:12] xs.len() = 3  ⟵ dbg! prints *file:line*
                                   the *expression* and its *value*

xs = [1, 2, 3]                   ⟵ println! prints what you choose
```

- **`println!`** - quick, but you **format it yourself**
- **`dbg!(expr)`** - prints **file:line**, the **expression** and its **value**, then **returns it**
- `rustc --explain E0382` - the **full story** behind an error code

Read the borrow-checker hint first - it usually names the fix.

Logging



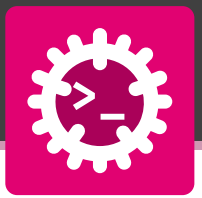
Beyond a quick print: **levels** you can switch on/off **without recompiling**.

```rust
use log::{info, warn, error};

fn main() {
    env_logger::init();
    info!("starting on port {}", 8080);
    warn!("connection slow, retrying");
    error!("giving up");
}
```

```
RUST_LOG=info cargo run    # show info and above, everywhere
RUST_LOG=debug cargo run   # show debug and above
RUST_LOG=trace cargo run   # everything
```

- levels: `error!` > `warn!` > `info!` > `debug!` > `trace!`
- `log` is a **facade** - pick a backend: `env_logger`, `tracing`
- **`tracing`** - structured spans, great for **async** / production
- filter via `RUST_LOG`, **no code change**

Logging

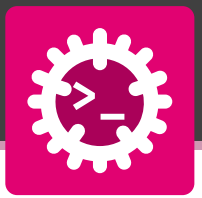


```
================= RUST_LOG=info =================
[2026-06-23T06:41:25Z INFO  logging_levels] application starting
[2026-06-23T06:41:25Z WARN  logging_levels::net] attempt 1 failed, retrying
[2026-06-23T06:41:25Z WARN  logging_levels::net] attempt 2 failed, retrying
[2026-06-23T06:41:25Z INFO  logging_levels::net] connected on attempt 3
[2026-06-23T06:41:25Z WARN  logging_levels] disk almost full: 92% used
[2026-06-23T06:41:25Z ERROR logging_levels] database connection lost
[2026-06-23T06:41:25Z INFO  logging_levels] application stopped
```

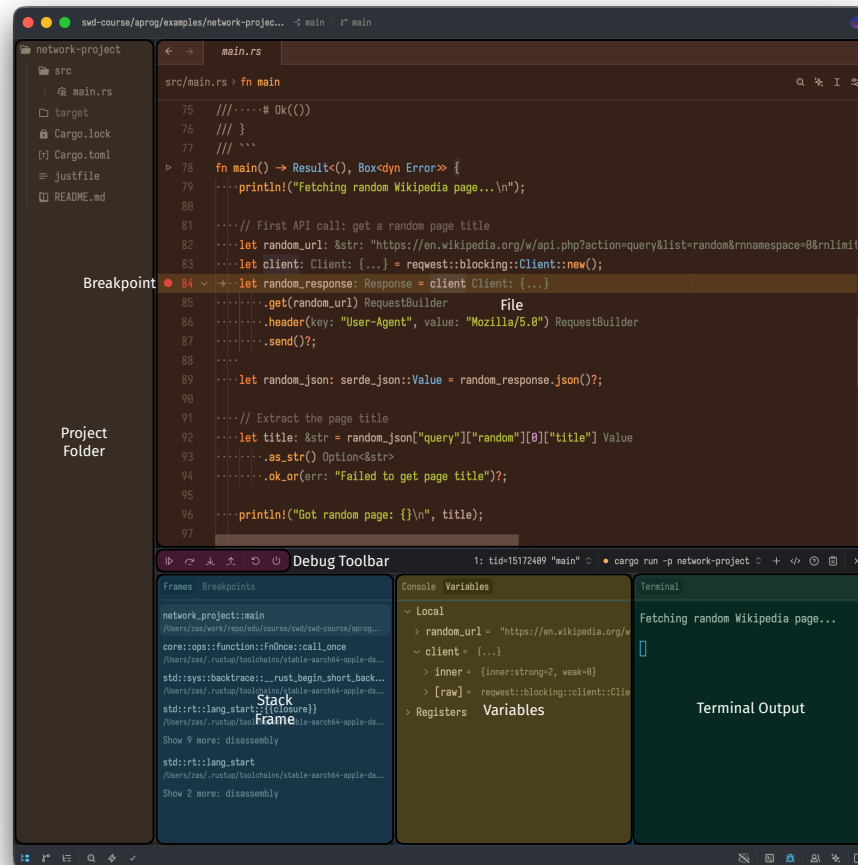
```
================= RUST_LOG=debug =================
[2026-06-23T06:41:25Z INFO  logging_levels] application starting
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 1/3
[2026-06-23T06:41:25Z WARN  logging_levels::net] attempt 1 failed, retrying
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 2/3
[2026-06-23T06:41:25Z WARN  logging_levels::net] attempt 2 failed, retrying
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 3/3
[2026-06-23T06:41:25Z INFO  logging_levels::net] connected on attempt 3
[2026-06-23T06:41:25Z WARN  logging_levels] disk almost full: 92% used
[2026-06-23T06:41:25Z ERROR logging_levels] database connection lost
[2026-06-23T06:41:25Z INFO  logging_levels] application stopped
```

```
================= RUST_LOG=trace =================
[2026-06-23T06:41:25Z INFO  logging_levels] application starting
[2026-06-23T06:41:25Z TRACE logging_levels::net] entering connect()
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 1/3
[2026-06-23T06:41:25Z WARN  logging_levels::net] attempt 1 failed, retrying
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 2/3
[2026-06-23T06:41:25Z WARN  logging_levels::net] attempt 2 failed, retrying
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 3/3
[2026-06-23T06:41:25Z INFO  logging_levels::net] connected on attempt 3
[2026-06-23T06:41:25Z WARN  logging_levels] disk almost full: 92% used
[2026-06-23T06:41:25Z ERROR logging_levels] database connection lost
[2026-06-23T06:41:25Z INFO  logging_levels] application stopped
```

Debugging with Breakpoints (in Zed / VS Code)



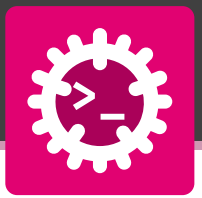
1. Open the *main.rs* file and click the *gutter* to set a *breakpoint*
2. run the program in *debug mode* F4
3. Go thought the program with the following actions:
   - *continue* - F5 - run until the next breakpoint
   - *step over* - F7 - run the current line
     stop at the next line
   - *step into* - Ctrl+F11 - run the current line
     stop at the first line of the called function
   - *step out* - Shift+F11 - run until the current function returns
4. **Inspect** live the:
   - *stack frame* - see the **call stack**
   - *variables* - see the **current values** of variables in scope
   - *terminal* - see the **program output**



Profiling

Make it fast

Benchmark & profile **Measure before you optimise.**



Hyperfine

`hyperfine` can benchmark a binary programs (warmups, stats, A/B compare)

```
hyperfine --warmup 3 'fd -e jpg -uu' 'find -iname "*.jpg"'
Benchmark 1: fd -e jpg -uu
  Time (mean ± σ):     15.0 ms ±   0.8 ms
          [User: 8.2 ms, System: 132.5 ms]
  Range (min … max): 13.0 ms …  17.7 ms    165 runs
```

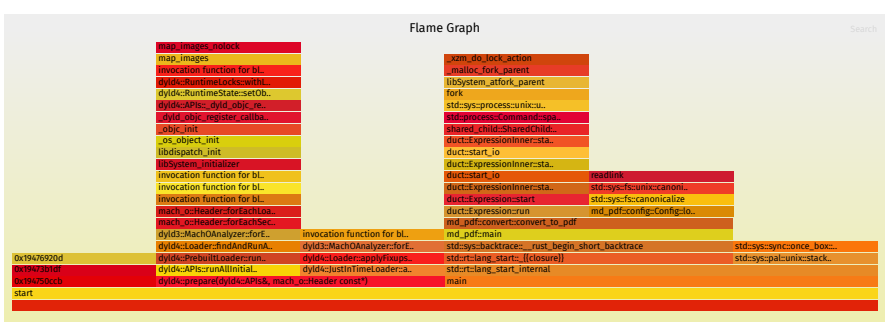
Read the split - wall-clock vs User / System Time

- *User* = your code on the Central Processing Unit (CPU)
- *System* = the kernel (syscalls, Input/Output (I/O))

Flamegraph

`cargo flamegraph` shows **where** the time is spent in your code, a **wider** frame is **hotter**

```
cargo flamegraph -o flamegraph.svg -- hyperfine -w 3
'fd -e jpg -uu' 'find -iname "*.jpg"'
```



Benchmark & profile **Measure before you optimise.**

Coming up: five **mystery programs** - guess what it does from the time **split** and the **flamegraph**.

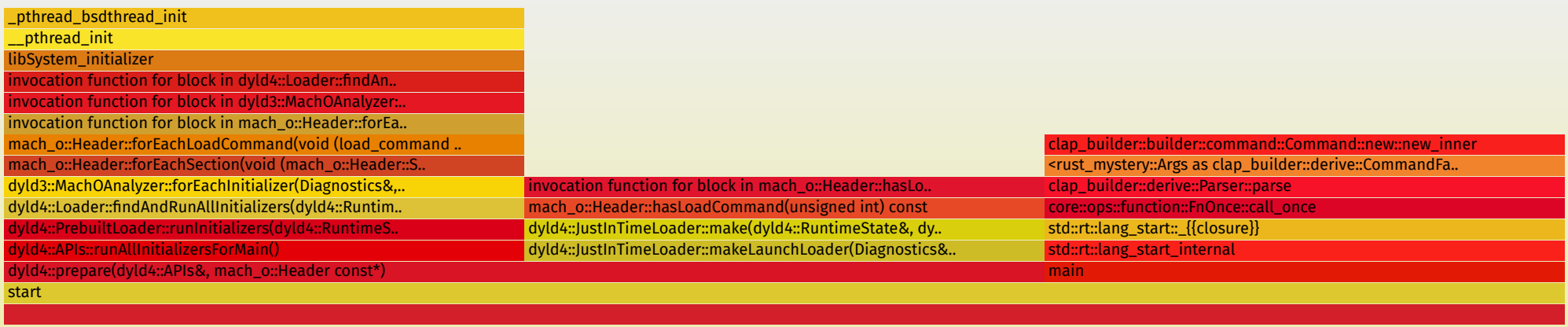
Mystery Program 1

```
hyperfine --warmup 3 --show-output --min-runs 10 "release/rust_mystery_v1_0_0_Mac_AARCH64 -m 1"
```

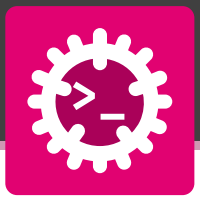
```
Execution Mystery Program 1
...
Time (mean ± σ):      2.013 s ±  0.002 s    [User: 0.003 s, System: 0.004 s]
Range (min … max):    2.009 s …  2.014 s    10 runs
```

Flame Graph

Search

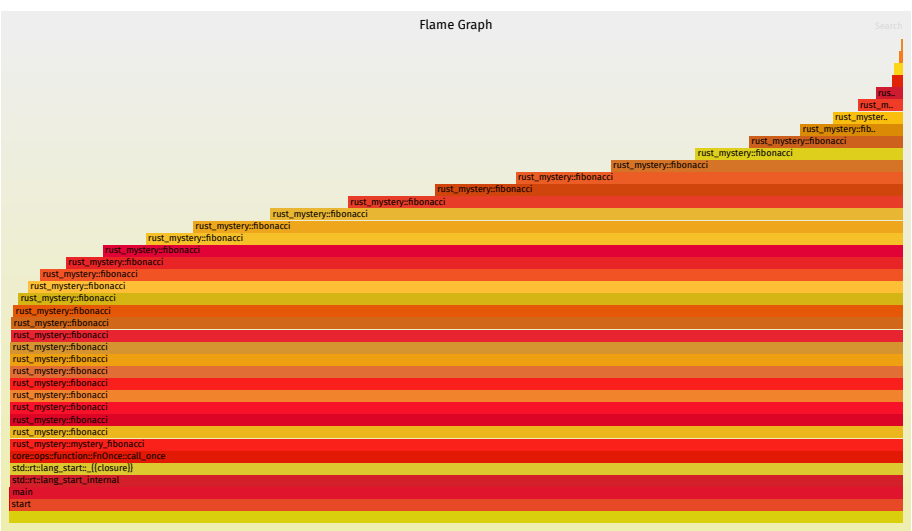


Mystery Program 2



```
hyperfine --warmup 3 --show-output --min-runs 10 "release/rust_mystery_v1_0_0_Mac_AARCH64 -m 2"
```

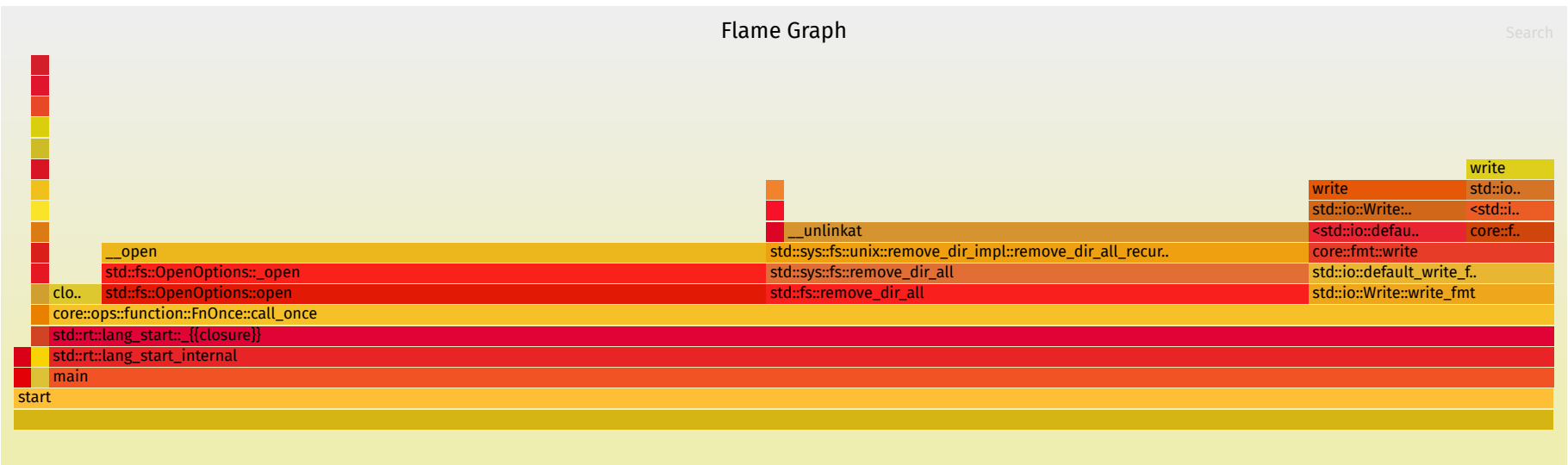
```
Execution Mystery Program 2
...
Time (mean ± σ):      2.213 s ±  0.009 s    [User: 2.192 s, System: 0.016 s]
Range (min … max):    2.201 s …  2.227 s    10 runs
```



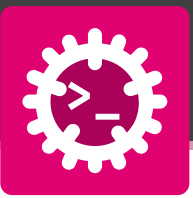
Mystery Program 3

```
hyperfine --warmup 3 --show-output --min-runs 10 "release/rust_mystery_v1_0_0_Mac_AARCH64 -m 3"
```

```
Execution Mystery Program 3
...
Time (mean ± σ):     115.0 ms ±  18.9 ms    [User: 2.5 ms, System: 107.6 ms]
Range (min … max):    83.0 ms … 143.0 ms    24 runs
```

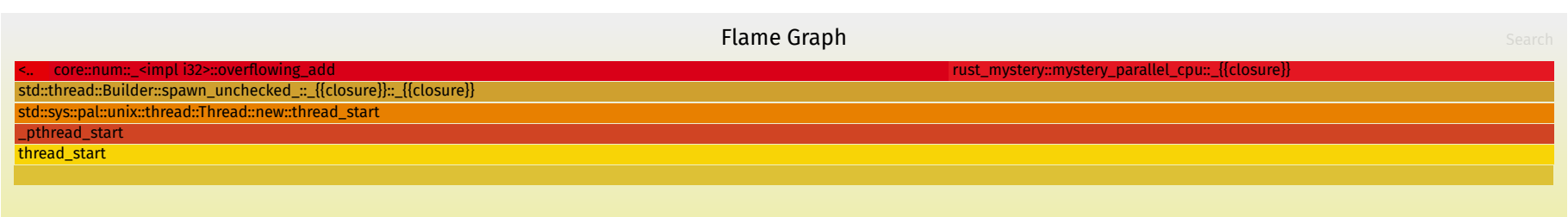


Mystery Program 4



```
hyperfine --warmup 3 --show-output --min-runs 10 "release/rust_mystery_v1_0_0_Mac_AARCH64 -m 4"
```

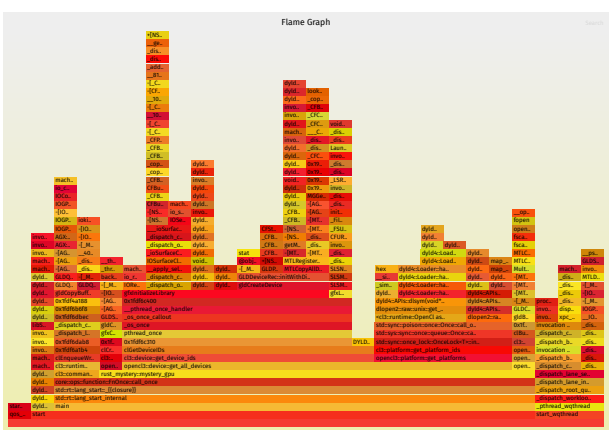
```
Execution Mystery Program 4
...
Time (mean ± σ):     760.5 ms ±  45.9 ms    [User: 10451.8 ms, System: 41.7 ms]
Range (min … max):   703.5 ms … 852.2 ms    10 runs
```



Mystery Program 5

```
hyperfine --warmup 3 --show-output --min-runs 10 "release/rust_mystery_v1_0_0_Mac_AARCH64 -m 5"
```

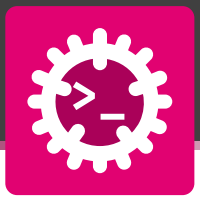
```
Execution Mystery Program 5
results front: 301
results back: 1300
kernel execution duration (ns): 204
...
  Time (mean ± σ):      47.4 ms ±   5.6 ms    [User: 11.0 ms, System: 13.5 ms]
  Range (min … max):    41.0 ms …  79.5 ms    62 runs
```



Documentation

Code others (and future-you) can read

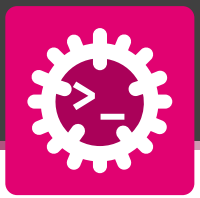
RustDoc comments



Doc comments become a **browsable website** - and the examples are **compiled and tested**.

```rust
//! # mycrate
//!
//! Temperature conversions for the AProg course.
//! Crate-level docs (`//!`) sit at the top of `lib.rs`
//! and become the front page of the generated site.
//!
//! # Examples
//! ```
//! use mycrate::temperature::Celsius;
//! assert_eq!(Celsius::new(0.0).to_fahrenheit(), 32.0);
//! ```

/// Temperature handling.
pub mod temperature {
  /// A temperature in degrees Celsius.
  ///
  /// # Examples
  /// ```
  /// use mycrate::temperature::Celsius;
  /// let t = Celsius::new(25.0);
  /// assert_eq!(t.to_fahrenheit(), 77.0);
  /// ```
  pub struct Celsius(f64);

  impl Celsius {
    /// Creates a new [`Celsius`] value.
    pub fn new(deg: f64) -> Self {
      Self(deg)
    }

    /// Converts to Fahrenheit.
    ///
    /// # Panics
    /// Never - this conversion is total.
    pub fn to_fahrenheit(&self) -> f64 {
      self.0 * 9.0 / 5.0 + 32.0
    }
  }
}
```

Doc comments: how they work



Documentation lives **next to the code** - the same source is both the comments and the published site.

Writing the comments

- `///` documents the **next item** - a struct, method, or function
- `//!` documents the **enclosing** item - the module or whole crate
- **Markdown** inside: headings, lists, code blocks
- convention sections: `# Examples`, `# Panics`, `# Errors`
- `[Celsius]` - an **intra-doc link** to another item

Building & testing

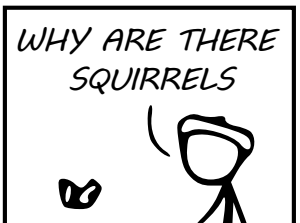
- the `# Examples` block is **real code**, compiled and run
- `cargo test` executes every doc example → docs **can't drift**
- public crates publish automatically to **docs.rs**

```
# build the HTML docs, open in browser
cargo doc --open

# also compiles & runs the doc examples
cargo test
```

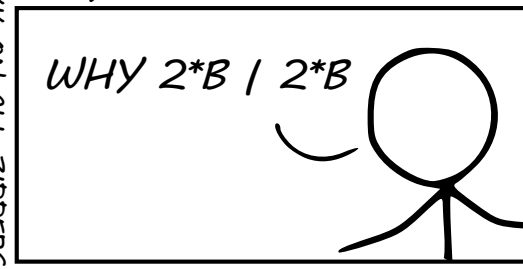
QUESTIONS

CAN BE ASKED BY ANYONE ANYTIME

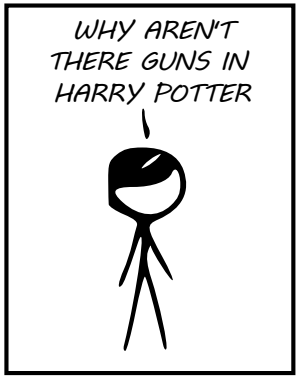
WHY DO WHALES JUMP
WHY ARE THERE MIRRORS ABOVE BEDS
WHY DO I SAY UH
WHY IS SEA SALT BETTER
WHY ARE THERE TREES IN THE MIDDLE OF FIELDS
WHY IS THERE NOT A POKEMON MMO
WHY IS THERE LAUGHING IN TV SHOWS
WHY ARE THERE DOORS ON THE FREEWAY
WHY ARE THERE SO MANY SVCHOST·EXE RUNNING
WHY AREN'T ANY COUNTRIES IN ANTARCTICA
WHY ARE THERE SCARY SOUNDS IN MINECRAFT
WHY IS THERE KICKING IN MY STOMACH
WHY ARE THERE TWO SLASHES AFTER HTTP
WHY ARE THERE CELEBRITIES
WHY DO SNAKES EXIST
WHY DO OYSTERS HAVE PEARLS
WHY ARE DUCKS CALLED DUCKS
WHY DO THEY CALL IT THE CLAP
WHY ARE KYLE AND CARTMAN FRIENDS
WHY IS THERE AN ARROW ON AANG'S HEAD
WHY ARE TEXT MESSAGES BLUE
WHY ARE THERE MUSTACHES ON CLOTHES
WHY WUBA LUBBA DUB DUB MEANING
WHY IS THERE A WHALE AND A POT FALLING
WHY ARE THERE SO MANY BIRDS IN SWISS
WHY IS THERE SO LITTLE RAIN IN WALLIS
WHY IS WALLIS WEATHER FORECAST ALWAYS WRONG
WHY ARE THERE MALE AND FEMALE BIKES
WHY ARE THERE BRIDESMAIDS
WHY DO DYING PEOPLE REACH UP
HOW FAST IS LIGHTSPEED
WHY ARE OLD KLINGONS DIFFERENT
WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPYDER BITES ITCH



WHY ARE HATS SO EXPENSIVE
WHY IS THERE CAFFEINE IN MY SHAMPOO
WHY HAVE DINOSAURS NO FUR
WHY DO TWINS HAVE DIFFERENT FINGERPRINTS
WHY ARE SWISS AFRAID OF DRAGONS
WHY IS HTTPS CROSSED OUT
WHY IS THERE A LINE THROUGH
WHY IS THERE A RED LINE THROUGH HTTPS
WHY IS HTTPS IMPOR

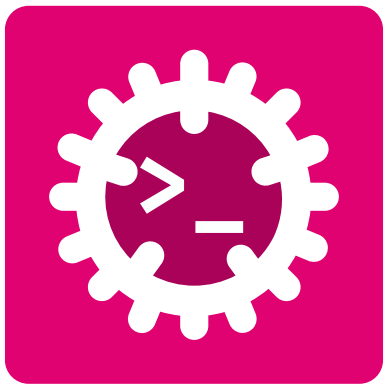


WHY AREN'T ECONOMISTS RICH
WHY DO AMERICANS CALL IT SOCCER
WHY ARE MY EARS RINGING
WHY IS 42 THE ANSWER TO EVERYTHING
WHY CAN'T NOBODY ELSE LIFT THORS HAMMER
WHY IS MARVIN ALWAYS SO SAD
WHY ARE THERE ANTS IN MY LAPTOP
WHY IS EARTH TILTED
WHY IS SPACE BLACK
WHY IS OUTER SPACE SO COLD
WHY ARE THERE PYRAMIDS ON THE MOON
WHY IS NASA SHUTTING DOWN
WHY IS THERE A SWARM OF ANTS
WHY IS THERE PILEGRIM
WHY ARE THERE SO MANY CROWS IN ROCHES
WHY IS TO BE OR NOT TO BE FU
WHY DO CHILDREN GET CANCER
WHY IS POSEIDON ANGRY WITH ODYSSEUS
WHY IS THERE ICE IN SPACE

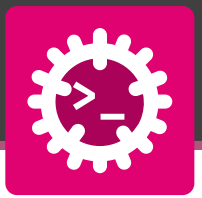


WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE FPGA's EVERYWHERE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOCKS
WHAT IS https://xkcd.com/1256/
WHY DO THEY SAY T-MINUS
WHY ARE OCEANS BECOMMING MORE ACIDIC
WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO PEOPLE DIE

WHY DO IGUANAS DIE
WHY AREN'T THERE DINOSAUR GHOSTS
WHY IS THERE LAVA
WHY IS THERE HE
WHY ARE THERE FEMA
WHY IS YKK ON ALL ZIPPERS



Glossary & Bibliography

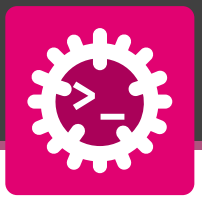


Computer Science

CPU – **Central Processing Unit**: The main processing component of a computer. 23

Concurrency

I/O – **Input/Output**: Operations that involve reading from or writing to external devices or resources. 23



Bibliography

[1] S. Klabnik, C. Nichols, and C. Krycho, “The Rust Programming Language.”

[2] J. Blandy, J. Orendorff, and L. F. s Tindall, *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, Incorporated, 2021.

[3] H. Collard, *Black Hat Rust*, vol. 1. Amazon, 2025.

[4] A. Shvets, “Refactoring Guru.” 2026.

[5] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education, 2008.

[6] “SVG Repo - Free SVG Vectors and Icons.”